SQL Interview Question Bank

200-Question Comprehensive Revision Bank | Target: MySQL 8+

This document contains a structured 200-question progression designed to build and test your SQL interview pattern recognition. Rather than testing isolated syntax, the progression forces the combination of concepts commonly seen in modern technical interviews.

Difficulty Progression

Level	Difficulty	Questions	Focus Areas
L1	Basic	1–25	SELECT, WHERE, DISTINCT, ORDER BY, Limits
L2	Basic +	26–50	Aggregates, Aliases, CASE, Dates, COALESCE
L3	Easy	51–75	GROUP BY vs HAVING
L4	Medium	76–110	INNER, LEFT, SELF JOINS, Multiple Table Logic
L5	Medium +	111–140	Subqueries, EXISTS, Set Operations (UNION)
L6	Hard	141–165	CTEs, Window Functions (RANK, ROW_NUMBER)
L7	Advanced	166–185	Complex Window Functions, Multi-Concept Joins
L8	Expert / Interview Patterns	186–200	Streaks, Gaps, Retention, Recursive CTEs

I. Database Schema (MySQL 8+)

All questions in this bank operate on the following normalized relational schema. Assume appropriate data types and populated records for testing.

```sql
-- 1. Departments & Locations
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL,
    location VARCHAR(50) NOT NULL
);

-- 2. Employees & Hierarchy
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    salary DECIMAL(10,2) NOT NULL,
    dept_id INT,
    manager_id INT,
    hire_date DATE NOT NULL,
    job_role VARCHAR(50),
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id),
    FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
);

-- 3. Historical/Set Operation Data
CREATE TABLE archived_employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    salary DECIMAL(10,2),
    dept_id INT,
    hire_date DATE
);

-- 4. Projects & Allocations
CREATE TABLE projects (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(100) NOT NULL,
    dept_id INT,
    budget DECIMAL(12,2),
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

CREATE TABLE employee_projects (
    emp_id INT,
    project_id INT,
    revenue_generated DECIMAL(12,2) DEFAULT 0.00,
    PRIMARY KEY (emp_id, project_id),
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id),
    FOREIGN KEY (project_id) REFERENCES projects(project_id)
);

-- 5. E-Commerce / Products & Sales
```

```sql
CREATE TABLE categories (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL
);

CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category_id INT,
    unit_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);

CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    city VARCHAR(50),
    joined_date DATE NOT NULL
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- 6. Logins (Streaks, Gaps & Retention)
CREATE TABLE user_logins (
    login_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    login_date DATE NOT NULL
);
```

II. Question Bank

L1 — SQL Fundamentals (Questions 1–25)

> **Focus:** `SELECT`, `WHERE`, operators, `DISTINCT`, `ORDER BY`, `LIMIT`, basic functions.

1. Retrieve all columns from the `employees` table.
2. Retrieve only employee names and salaries.
3. Retrieve all unique department IDs.
4. Find employees whose salary is greater than 50,000.
5. Find employees whose salary is exactly 60,000.
6. Find employees whose salary is less than 40,000.
7. Find employees who belong to department 10.
8. Find employees who do not belong to department 10.
9. Find employees whose salary is between 40,000 and 70,000.
10. Find employees whose department is either 10, 20, or 30.
11. Find employees whose name starts with 'A'.
12. Find employees whose name ends with 'n'.
13. Find employees whose name contains 'an'.
14. Find employees whose manager ID is `NULL`.
15. Find employees who have a manager.
16. Sort employees by salary in ascending order.
17. Sort employees by salary in descending order.
18. Sort employees first by department and then by salary descending.
19. Find the five highest-paid employees.
20. Find the five lowest-paid employees.
21. Count the total number of employees.
22. Find the highest salary.
23. Find the lowest salary.
24. Find the average salary.
25. Find the total salary paid to all employees.

L2 — Basic SQL + Functions (Questions 26–50)

> **Focus:** aggregate functions, aliases, expressions, `CASE`, strings, dates, `COALESCE`.

1. Display employee name and salary with the salary increased by 10%.
2. Display employee name and annual salary.
3. Display employee name and a calculated column showing whether the employee earns more than 60,000.
4. Find the number of employees who earn more than 50,000.
5. Find the number of employees who belong to department 20.
6. Find the average salary of employees in department 10.
7. Find the total salary paid to employees in department 30.
8. Find the difference between the highest and lowest salary.
9. Count the number of employees whose manager ID is not `NULL`.
10. Count the number of distinct departments.
11. Display employee names in uppercase.
12. Display employee names in lowercase.
13. Find the length of each employee's name.
14. Find employees whose names contain exactly five characters.
15. Extract the year from each employee's joining date.
16. Find employees who joined after January 1, 2023.
17. Find employees who joined during 2024.
18. Find the number of employees who joined each year.
19. Display `NULL` manager IDs as 'No Manager'.
20. Categorize employees as Low, Medium, or High salary using `CASE`.
21. Find the average salary rounded to two decimal places.
22. Find employees whose salary is an even number.
23. Find employees whose salary is divisible by 5,000.
24. Find the three most recently joined employees.
25. Find employees who joined in the current month.

L3 — GROUP BY & HAVING (Questions 51–75)

> **Focus:** aggregation and understanding the difference between `WHERE` and `HAVING`.

1. Find the number of employees in each department.
2. Find the average salary in each department.
3. Find the maximum salary in each department.
4. Find the minimum salary in each department.
5. Find the total salary paid by each department.
6. Find the number of employees hired in each year.
7. Find the number of employees hired in each month.
8. Find departments having more than five employees.
9. Find departments whose average salary exceeds 60,000.
10. Find departments whose total salary exceeds 500,000.
11. Find departments having at least one employee earning more than 100,000.
12. Find departments where the minimum salary is greater than 40,000.
13. Find departments where the maximum salary is less than 100,000.
14. Find the number of employees for each combination of department and job role.
15. Find the average salary for each job role.
16. Find job roles having more than three employees.
17. Find job roles whose average salary exceeds the company-wide average salary.
18. Find the department with the highest average salary.
19. Find the department with the lowest average salary.
20. Find the department with the highest total salary expenditure.
21. Find the department with the most employees.
22. Find the salary range (MAX - MIN) for every department.
23. Find departments where the salary range exceeds 50,000.
24. Find the year with the highest number of employee hires.
25. Find departments where more than 50% of employees earn above 60,000.

L4 — JOINS (Questions 76–110)

> **Focus:** `INNER JOIN`, `LEFT JOIN`, self joins, multiple joins.

1. Display each employee along with their department name.
2. Display each employee along with their manager's name.
3. Find employees who do not belong to any department.
4. Find departments that currently have no employees.
5. Find employees whose department name is 'IT'.
6. Find employees working in departments located in 'Hyderabad'.
7. Find the number of employees in each department using a join.
8. Find the average salary of each department using a join.
9. Find the highest-paid employee in each department.
10. Find employees earning more than 60,000 along with their department names.
11. Find employees who earn more than their manager.
12. Find employees who earn less than their manager.
13. Find employees who have the same salary as their manager.
14. Find employees who report directly to a particular manager.
15. Find managers who have more than five employees.
16. Find departments and their total salary expenditure.
17. Find departments with no employees using a `LEFT JOIN`.
18. Find customers who have placed at least one order.
19. Find customers who have never placed an order.
20. Find the total number of orders placed by each customer.
21. Find customers who have placed more than five orders.
22. Find the total amount spent by each customer.
23. Find customers whose total spending exceeds 100,000.
24. Find products that have never been ordered.
25. Find the most frequently ordered product.
26. Find the total quantity sold for each product.
27. Find the total revenue generated by each product.
28. Find the top five products by revenue.
29. Find employees assigned to more than three projects.

30. Find projects with more than five employees assigned.
31. Find employees who are not assigned to any project.
32. Find projects that currently have no employees.
33. Find employees working on projects managed by their own department.
34. Find the department whose employees have generated the highest project revenue.
35. Find the customer who has spent the most money on products.

L5 — Subqueries, EXISTS & Set Operations (Questions 111–140)

> **Focus:** Correlated subqueries, `EXISTS`, `NOT EXISTS`, `UNION`, `INTERSECT`, `EXCEPT`.

1. Find employees earning more than the average salary.
2. Find employees earning the maximum salary.
3. Find employees earning the minimum salary.
4. Find employees earning more than the average salary of department 10.
5. Find employees whose salary is greater than their department's average salary.
6. Find departments whose average salary is greater than the company average.
7. Find employees working in the department with the highest average salary.
8. Find the second-highest salary.
9. Find the third-highest salary.
10. Find the nth-highest distinct salary.
11. Find employees who earn the second-highest salary.
12. Find departments containing at least one employee earning more than 100,000.
13. Find departments where every employee earns more than 40,000.
14. Find customers who have placed at least one order using `EXISTS`.
15. Find customers who have never placed an order using `NOT EXISTS`.
16. Find products that have never appeared in an order.
17. Find employees who work in departments located in Hyderabad.
18. Find employees who earn more than every employee in department 20.
19. Find employees who earn more than at least one employee in department 20.
20. Find departments whose employee count is greater than the average department employee count.
21. Find the employee with the highest salary in each department using a correlated subquery.
22. Find employees whose salary is greater than their manager's salary using a correlated subquery.

23. Find customers whose spending is greater than the average customer spending.
24. Find products whose revenue is greater than the average product revenue.
25. Combine employee names from two employee tables using `UNION`.
26. Combine two employee datasets while preserving duplicates using `UNION ALL`.
27. Find records appearing in both datasets using `INTERSECT`.
28. Find records appearing in the first dataset but not the second using `EXCEPT`.
29. Find customers who purchased both Product A and Product B.
30. Find customers who purchased Product A but never purchased Product B.

L6 — CTEs + Window Functions (Questions 141–165)

> **Focus:** `WITH` clause, `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `LAG()`, `LEAD()`.

1. Create a CTE containing employees earning more than 60,000.
2. Use a CTE to calculate average salary by department.
3. Use a CTE to find departments whose average salary exceeds the company average.
4. Use multiple CTEs to calculate customer spending and compare it with average spending.
5. Use a CTE to calculate total revenue by product and return the top five products.
6. Assign a row number to employees ordered by salary.
7. Assign row numbers separately within each department.
8. Rank employees by salary.
9. Compare `RANK()` and `DENSE_RANK()` for employees with duplicate salaries.
10. Find the highest-paid employee in each department using `ROW_NUMBER()`.
11. Find the top three highest-paid employees in each department.
12. Find the second-highest-paid employee in each department.
13. Find the third-highest-paid employee in each department.
14. Find employees whose salary is above their department's average using a window function.
15. Calculate the running total of salaries ordered by joining date.
16. Calculate cumulative revenue by order date.
17. Calculate the average salary of the current employee's department using a window function.
18. Find the difference between each employee's salary and their department average.
19. Find the percentage contribution of each employee's salary to their department's total salary.
20. Use `LAG()` to display each employee's previous salary based on joining order.

21. Use `LEAD()` to display the next employee's salary.
22. Find employees whose salary is higher than the previous employee's salary.
23. Find month-over-month sales growth.
24. Find the first order placed by every customer.
25. Find the latest order placed by every customer.

L7 — Hard Interview Patterns (Questions 166–185)

> **Focus:** Combining multiple SQL concepts (Window Functions + Joins + CTEs).

1. Find the top three salaries in every department, including ties.
2. Find the highest-paid employee in every department along with the department average salary.
3. Find employees whose salary is greater than both their manager's salary and their department's average salary.
4. Find departments where the highest-paid employee earns more than twice the department average.
5. Find the department with the largest difference between its highest and lowest salaries.
6. Find the second-highest salary in every department without using `LIMIT`.
7. Find employees who have the same salary as another employee in their department.
8. Find duplicate employee records based on name, email, and department.
9. Find customers who placed orders in every month of a particular year.
10. Find customers who purchased every product in a particular category.
11. Find the product category generating the highest revenue.
12. Find the customer who has purchased the largest number of distinct products.
13. Find the customer who placed the longest sequence of consecutive daily orders.
14. Find the longest period during which a customer made no purchases.
15. Find the first and last order date for every customer and calculate the number of days between them.
16. Find customers whose latest order amount is greater than their average order amount.
17. Find products whose sales increased every month for three consecutive months.
18. Find employees whose salary has increased compared with the previous employee hired in the same department.
19. Find the percentage of employees in each department whose salary is above the company average.
20. Find the department contributing the largest percentage of the company's total salary expenditure.

L8 — Advanced / Expert Interview Problems (Questions 186–200)

> **Focus:** Streaks, gaps, cohort retention, recursive CTEs, and median calculation.

1. Find the longest consecutive login streak for every user.
2. Find all users who logged in for at least five consecutive days.
3. Find gaps in a customer's order history where the gap between two orders exceeds 30 days.
4. Find users who made purchases in three consecutive months.
5. Find the month with the highest month-over-month revenue growth.
6. Find the top 10% of employees by salary within each department.
7. Find the median salary of every department.
8. Find the median order value for every customer.
9. Find the products whose cumulative revenue accounts for the first 80% of total revenue.
10. Find customers whose first purchase was also their highest-value purchase.
11. Find the percentage of customers who made another purchase within 30 days of their first purchase.
12. Find the retention rate of customers month-over-month based on their purchase activity.
13. Find the best-selling product in every category for every month.
14. Find the employee hierarchy starting from the CEO and display every employee's level in the hierarchy using a recursive CTE.
15. Given an employee hierarchy, calculate the total number of employees and total salary under every manager, including indirect reports.